

Code Design Test: Data Compression Design

Explanation:

The solution for the *Data Compression Design Task* takes advantage of the assumptions detailed in the problem set. Specifically, this refers to the fact that it is common for data in the buffer to be repeated in series and that each byte contains a number from 0 to 127 (0x00 to 0x7F).

So, a simple way to compress the values in the buffer is to remove the redundant repeated values and instead represent them as a count and the data itself. For example, the values [..., 0x04, 0x04, x04, ...] could be represented as [..., 0x03, 0x04, ...] where 0x03 is the **count**, and 0x04 is the data. For long repetitions, this dramatically reduces the size of the buffer. However, if the buffer contains many single values (i.e. no repetitions), this approach can actually *increase* the size of the buffer, as a single value would be represented by a count of 1 followed by the data. Clearly, this isn't ideal.

Therefore, a way to overcome this limitation is to leave the non-repeated values untouched and only add (count/value) pairs for repetitions of two or more. This is where the assumption that each byte contains 0x00 to 0x7F comes in. Under this assumption, the most significant bit of the byte is **always** 0. So, we can use this to our advantage. A count value will then be represented by a byte with the most significant bit set to 1, and a data value with the most significant bit set to 0. In the example first described above, [..., 0x04, 0x04, x04, ...] is now represented as [..., 0x83, 0x04, ...] where 0x83 describes the **count**, and 0x04 is the data.

When compressing, this reduces to ORing the repeated count with 0b1000 000 (0x80) before storing. When decompressing, this means we must first scan this most significant bit to see if we are dealing with a count or a data value, before taking the appropriate action to decompress the buffer.

Example data and function call:

All this being said, given the example data from the question:

```
// Data before the call
// data_ptr[] = { 0x03, 0x74, 0x04, 0x04, 0x04, 0x35, 0x35, 0x64,
//               0x64, 0x64, 0x64, 0x00, 0x00, 0x00, 0x00, 0x00,
//               0x56, 0x45, 0x56, 0x56, 0x56, 0x09, 0x09, 0x09 };
// data_size = 24;
```

The compressed data becomes:

```
// Data after the call
// data_ptr[] = { 0x03, 0x74, 0x83, 0x04, 0x82, 0x35, 0x84, 0x64,
//               0x85, 0x00, 0x56, 0x45, 0x83, 0x56, 0x83, 0x09 };
// data_size = 16;
```

Complexity:

The implementation for *byte_compress()* runs in **O(n)**, as it needs a single pass through all the values in the buffer to spot repetitions.